BSDA2001P: Introduction to DL and GenAI Project

Smart MCQ Solver Using Deep Learning and Pretrained Transformers

- Course: BSDA2001P - Introduction to DL and GenAI Project
- Project Title: Smart MCQ Solver Challenge
- Student Name: Pratham Shah
- Roll No.: 23f2003391
- Submission Date: 8th Aug 2026

Table of Contents

1. Abstract / Executive Summary
2. Introduction
3. Dataset & Preprocessing
4. Tokenization Strategy
5. Modeling & Experimentation
6. Performance & Comparative Analysis
7. Conclusion & Future Work
8. References

1. Abstract / Executive Summary

This project focuses on the Smart MCQ Solver Challenge on Kaggle. Each question contains five options labelled A-E, and the task is to rank the three most likely answers. The competition uses Mean Average Precision at 3, or MAP@3, as the evaluation metric.

I started with TF-IDF and frozen BERT logistic regression baselines. I then trained a BiLSTM from scratch and finetuned DeBERTa-v3-small and MiniLM as multiple choice models. I also combined DeBERTa and MiniLM using a weighted probability ensemble.

The most important finding from EDA was that the dataset contained a large number of repeated questions. The 2,000 training rows represented only 252 normalized questions after removing prompt prefixes and suffixes. Because of this, I used five fold question level validation instead of a random row split.

The listwise BiLSTM achieved the best Kaggle MAP@3 of 0.7510. DeBERTa achieved 0.7448, MiniLM achieved 0.7427, and the weighted DeBERTa–MiniLM ensemble achieved 0.7472. DeBERTa had the strongest grouped OOF score of 0.6483.

Overall, this project showed me that understanding the dataset and designing the validation split correctly can matter as much as the model architecture.

BSDA2001P: Introduction to DL and GenAI Project

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge requires the model to rank five possible answers for each question. The final submission contains an id column and a prediction column. The prediction is a sequence such as C A E, where the first option is the model's most likely answer.

The competition uses MAP@3. If the correct answer is ranked first, the score for that question is 1. If it is ranked second, the score is 1/2, and if it is ranked third, the score is 1/3. No score is given if the correct answer is not within the top three.

This makes the problem different from normal multiclass classification because the ranking of the options matters, not just the top prediction.

2.2 Project Objective

My main objective was to compare different ways of solving the same MCQ ranking problem. I wanted to understand how classical text features, a model trained from scratch, and pretrained transformers behave on this dataset.

- Build TF-IDF and frozen BERT baselines.
- Train a BiLSTM completely from scratch.
- Finetune DeBERTa and MiniLM.
- Track training and validation metrics using W&B.
- Combine different models using probability averaging.

3. Dataset & Preprocessing

3.1 Dataset Description

The training dataset contains 2,000 rows. Each row has an id, a prompt, five answer options from A to E, and the correct answer. The test dataset contains 500 rows with the same columns except for the answer.

I did not find any missing values in either dataset.

The answer distribution was:

- A: 369
- B: 490
- C: 459
- D: 358
- E: 324

The classes are not perfectly balanced, but the difference is not very large. B and C are slightly more common, while E is the least common answer.

3.2 Exploratory Data Analysis (EDA)

The average prompt length was approximately 118 characters. I also compared the length of the correct option with the incorrect options.

Correct options contained an average of 28.66 words, while incorrect options contained an average of 25.68 words. The longest option was correct in roughly 35% of the questions. This was an interesting pattern, but it was not strong enough to use as a prediction rule.

The more important discovery was that the prompts contained repeated prefixes and suffixes. For example, the same question could appear with prefixes such as “Choose the correct answer” or “Select the most accurate option.”

3.3 Repeated Questions

Only 242 prompts were exact duplicates. However, after removing the repeated prefixes and suffixes, I found that 1,748 rows were duplicates. This reduced the training dataset to only 252 normalized questions.

Some questions appeared once, while others appeared close to 20 times. The correct answer text could also change slightly across repeated versions of the same question.

The test set contained 228 normalized questions. All 500 test rows matched a normalized question from the training data, and 455 test rows also matched an option set seen in training.

This helped explain why relatively simple models performed well on Kaggle. The public test set was much closer to the training data than the grouped validation folds.

3.4 Data Preprocessing

I created split_prompt() to remove the repeated prefix and suffix from each prompt. The function handles prompts with a colon and prompts containing one or two question marks.

After extracting the main question, clean_text() converts the text to lowercase and removes extra whitespace. I kept punctuation and digits because they can be important in technical questions.

For the BiLSTM, I split the cleaned text using whitespace. The question and answer option were joined using a special <SEP> token.

BSDA2001P: Introduction to DL and GenAI Project

4. Tokenization Strategy

4.1 Primary Tokenizer Selection: Scratch Tokenizer

The BiLSTM builds a new vocabulary inside each fold using only that fold's training rows. <PAD>, <UNK> and <SEP> have fixed IDs 0, 1 and 2.

Question and option tokens are concatenated/truncated to 100 tokens and right padded.

Fold specific vocabulary is used as it prevents validation text from influencing token IDs or embeddings hence preventing leakage.

100 tokens were taken as max_length as 99th percentile for token length was 100(max was ~148).

4.2 Transformer tokenization

For DeBERTa, I used the tokenizer associated with microsoft/deberta-v3-small. MiniLM used the tokenizer provided with sentence-transformers/all-MiniLM-L6-v2.

Each question was paired separately with its five options. This produced five sequences for every MCQ. The tokenizer applied padding and truncation with max_length=100.

The final transformer input had the shape: (batch_size, 5, 100)

Attention masks were also created so that the models could distinguish actual tokens from padding.

DeBERTa used a batch size of 8 because it was the larger model. MiniLM was smaller and could use a batch size of 32.

4.3 Leakage aware five fold validation

A random row split was not suitable because repeated forms of the same question could appear in both training and validation.

Instead, I used the normalized question, q_clean, as the grouping key. The unique questions were shuffled using seed 33 and divided into five folds. All rows belonging to one normalized question were kept in the same fold.

This removed direct normalized question overlap between training and validation.

This method is still not perfect. Questions with small semantic changes, such as replacing one noun with another, may remain in different groups. However, it is much safer than a normal row level split.

BSDA2001P: Introduction to DL and GenAI Project

5. Modeling & Experimentation

5.1 Model 1: Fine Tuned DeBERTa-v3-small

The main pretrained model was microsoft/deberta-v3-small. DeBERTa represents the content and relative position of tokens separately in its attention mechanism. This helps it compare the relationship between a question and closely related answer options.

I used AutoModelForMultipleChoice from Hugging Face. For each question, the model receives five question option pairs and produces five logits, one for each answer.

The input has shape: (batch_size, 5, 100)

The correct answer is stored as a class index from 0 to 4. I used cross entropy loss to train the model.

All pretrained layers were finetuned. I did not freeze any layers.

The main settings were:

- Model: microsoft/deberta-v3-small
- Batch size: 8
- Learning rate: 2e-5
- Optimizer: AdamW
- Weight decay: 0.01
- Maximum epochs: 15
- Early stopping patience: 5
- Cross validation folds: 5

The final grouped OOF MAP@3 was 0.6483, and the Kaggle MAP@3 was 0.7448.

5.2 Model 2: Custom Deep Learning Model BiLSTM

The BiLSTM was trained completely from scratch. Its vocabulary, word embeddings, recurrent weights and classifier were initialized without using pretrained language model weights.

Each question was paired with all five answer options. The model processed the options separately and then returned five logits for the complete question.

Stage	Shape	Operation
Grouped input	(B, 5, 100)	Five tokenized question option sequences
Flatten	(B×5, 100)	Treat each option as one sequence

BSDA2001P: Introduction to DL and GenAI Project

Embedding	(B×5, 100, 128)	Learned lookup + dropout 0.3
BiLSTM	(B×5, 100, 256)	1 layer, hidden 128 each direction
Masked mean	(B×5, 256)	Average non padding time steps
Classifier	(B×5, 1)	256 -> 128, LayerNorm, ReLU, dropout, 128 -> 1
Reshape	(B, 5)	Five jointly comparable option logits

The model uses a 128 dimensional embedding layer followed by a one layer bidirectional LSTM. Each LSTM direction has a hidden size of 128, giving an output size of 256.

I applied masked mean pooling over the non padding tokens. The pooled vector was passed through a classifier containing a linear layer, LayerNorm, ReLU, dropout and a final scoring layer.

Why I Changed BCE to Cross Entropy?

My original BiLSTM treated every question option pair as an independent binary example. This created one positive option and four negative options for each question, so I used binary cross entropy with pos_weight.

Later, I changed the model to output all five option scores together. The correct answer was represented as one class from 0 to 4, and the model was trained using cross entropy loss.

This formulation matched the actual MCQ structure better because exactly one answer is correct. It also removed the need for pos_weight.

The original pairwise BiLSTM achieved a Kaggle MAP@3 of 0.710. The final five fold listwise BiLSTM achieved an OOF MAP@3 of 0.5774 and a Kaggle MAP@3 of 0.7510.

5.3 Model 3: MiniLM Transformer

The second pretrained model was sentence-transformers/all-MiniLM-L6-v2. MiniLM is smaller than DeBERTa and uses six transformer layers with a hidden size of 384.

I loaded it using AutoModelForMultipleChoice and fine tuned the complete model. It used the same five option cross entropy formulation as DeBERTa.

The main settings were:

- Batch size: 32
- Learning rate: 5e-5
- Weight decay: 0.01

- Maximum epochs: 15
- Early stopping patience: 5
- Cross validation folds: 5

MiniLM achieved an OOF MAP@3 of 0.5369 and a Kaggle MAP@3 of 0.7427.

Although its grouped validation score was lower than DeBERTa, its Kaggle performance was close. This again shows how strongly the Kaggle test distribution overlaps with the training data.

5.4 Model 4: DeBERTa-MiniLM ensemble

For each transformer, I first averaged the test probabilities produced by its five fold models.I then combined the DeBERTa and MiniLM probabilities using:

final_probability = 0.3 × MiniLM + 0.7 × DeBERTa

I tested weights in steps of 0.05 using OOF predictions. The 0.3/0.7 combination gave the best result.

The weighted ensemble achieved an OOF MAP@3 of 0.6464 and a Kaggle MAP@3 of 0.7472. This was only a small improvement over the individual models. DeBERTa and MiniLM often made similar predictions, so averaging them did not add as much diversity as I initially expected.

5.5 Logistic Regression with Tfidf

TF-IDF with logistic regression was my first classical baseline.

Its initial validation score of 0.9893 was invalid because the vectorizer had been fitted before cross validation. This allowed information from the validation text to influence the features.

Despite this issue, the model still achieved a Kaggle MAP@3 of 0.7472. The likely reason is the large overlap between training and test questions rather than genuine semantic understanding.

This baseline was still useful because it showed how much of the dataset could be solved using lexical overlap.

5.6 Logistic Regression with BERT Pooling

I expected frozen BERT embeddings to improve over TF-IDF by providing contextual representations. I extracted BERT CLS embeddings and trained logistic regression on top of them.

Instead, the model achieved a lower Kaggle MAP@3 of 0.6217.

BSDA2001P: Introduction to DL and GenAI Project

One possible explanation is that the frozen embeddings were not adapted to distinguish between answer options that were semantically very similar.

6. Performance & Comparative Analysis

6.1 Evaluation metrics

MAP@3 was the main evaluation metric because it directly measures the ranked top three predictions required by the competition.

I also tracked training loss, validation loss and validation MAP@3 during training. The checkpoint with the best validation MAP@3 was saved for each fold.

Accuracy and F1 were not available for every historical run, so I did not include them in the final model comparison.

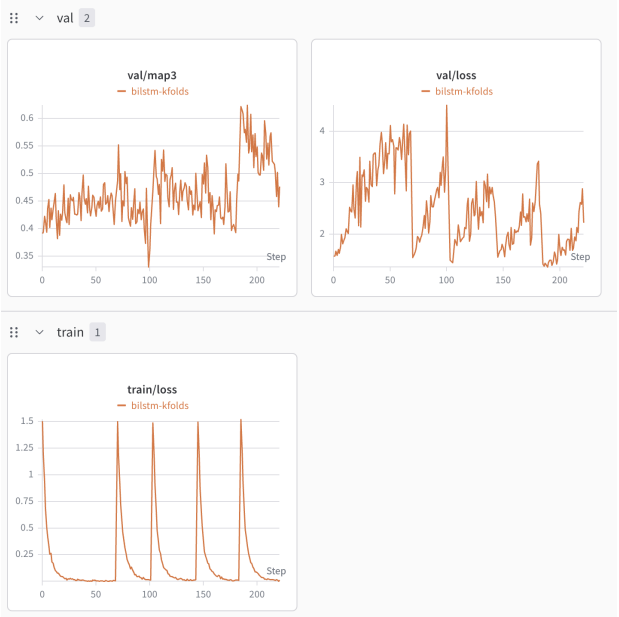
6.2 Training Performance & Comparative Report

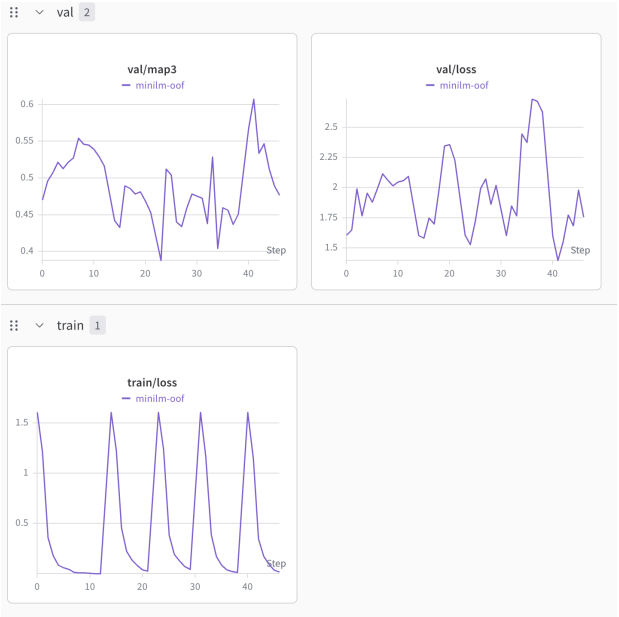
The W&B plots show training loss, validation loss and validation MAP@3 for the main deep learning models. The sudden resets in training loss occur because each fold starts with a newly initialized model and all folds are logged in the same run.

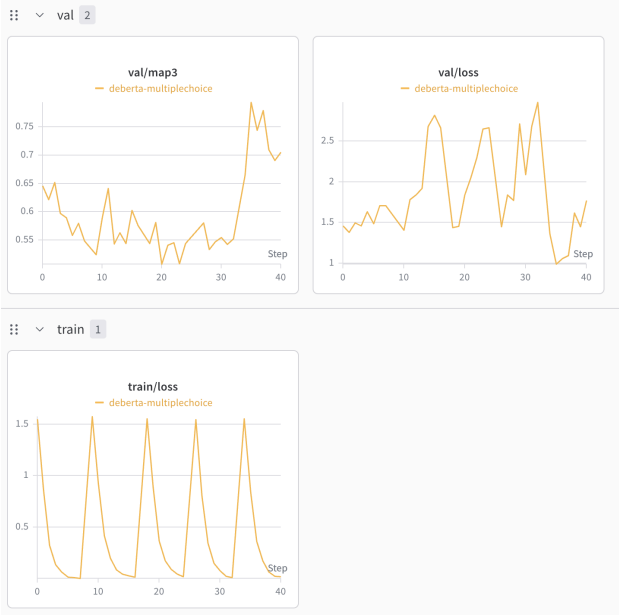
For DeBERTa and MiniLM, training loss decreased quickly, while validation loss became unstable after the first few epochs. This showed that the pretrained models could overfit the small number of unique question groups. Early stopping and MAP@3 based checkpoint selection helped reduce this problem.

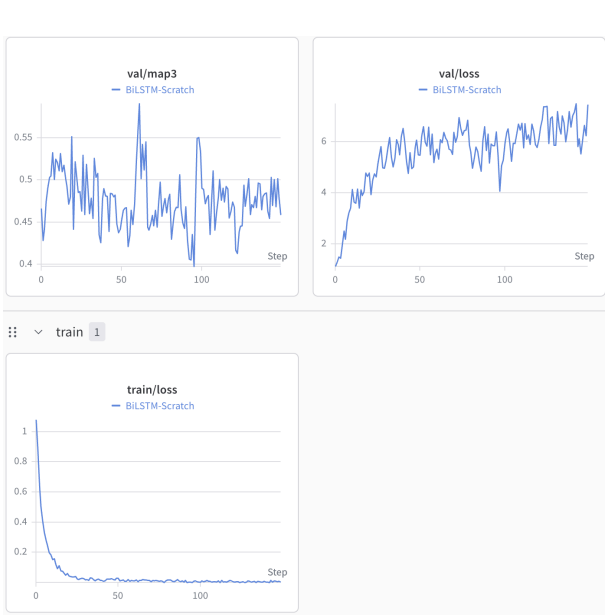
The BiLSTM trained more slowly, but it showed the same general pattern: training loss continued to decrease even when validation performance stopped improving.

BSDA2001P: Introduction to DL and GenAI Project









System	Validation evidence	Kaggle MAP@3	Interpretation
TF–IDF + LR	**0.9893 (Leakage)**	**0.7472**	**Strong lexical overlap baseline**
BERT + LR	**0.7568 (Leakage)**	**0.6217**	**Similar options**

BSDA2001P: Introduction to DL and GenAI Project

Original pairwise BiLSTM	**Val 0.5903**	**0.710**	**Independent BCE options**
Listwise scratch BiLSTM	**OOF 0.5774**	**0.7510**	**Best recorded score**
DeBERTa custom scorer	**OOF 0.6478**	**0.7414**	**Manual encoder + head**
DeBERTa AutoMC	**OOF 0.6483**	**0.7448**	**AutoModelForMultipleChoice**
MiniLM AutoMC	**OOF 0.5369**	**0.7427**	**Compact and efficient**
0.3 MiniLM + 0.7 DeBERTa	**OOF 0.6464**	**0.7472**	**Small gain from correlated models**

6.3 Kaggle Performance

Final Kaggle Score MAP@3: 0.75103
Ranking/Public Leaderboard: 689

7. Conclusion & Future Work

7.1 Key Learnings:

The biggest lesson from this project was that validation design mattered more than I initially expected.

At first, the dataset appeared to contain 2,000 different questions. After cleaning the prompt wrappers, I found that it contained only 252 normalized questions. This explained why random validation and simple lexical models could produce unusually high scores.

The scratch BiLSTM achieved the best public Kaggle result, while DeBERTa achieved the best grouped OOF result. This suggests that the models were benefiting from different properties of the dataset.

Changing the BiLSTM from independent binary scoring to a five option cross entropy objective also made the model easier to understand and better matched the actual competition task.

7.2 Challenges Faced:

The main challenges were:

- Repeated questions: A random row split produced unreliable validation because repeated questions appeared in both training and validation.
- Semantic near duplicates: Exact q_clean grouping does not catch every paraphrased version of a question.
- Transformer overfitting: DeBERTa and MiniLM training loss decreased rapidly while validation loss increased.
- Limited ensemble diversity: DeBERTa and MiniLM often made similar predictions, so the ensemble improvement was small.

7.3 Areas for Improvement:

If I continued the project, I would focus on three improvements.

- I would group semantically similar questions using sentence embeddings instead of relying only on exact normalized text. This would provide a stricter validation split.
- I would perform a proper error analysis by separating cases where the correct answer was ranked first, second, third or outside the top three. I would also compare the questions where DeBERTa, MiniLM and BiLSTM disagreed.
- I would try a more diverse ensemble by combining DeBERTa with the scratch BiLSTM or TF-IDF baseline. These models use different types of information and may complement each other better than DeBERTa and MiniLM.
- I would also compare different sequence lengths, use dynamic padding and repeat the experiments with multiple random seeds.

7.4 Conclusion

This project compared classical machine learning, a recurrent model trained from scratch and finetuned pretrained transformers for ranked MCQ solving.

The final listwise BiLSTM achieved the highest Kaggle MAP@3 of 0.7510. DeBERTa achieved the strongest grouped OOF result of 0.6483, while MiniLM and the transformer ensemble remained competitive.

The results showed that model complexity alone did not determine performance. The repeated question structure of the dataset, the validation method and the match between the training objective and competition metric had a major effect on the final results.

8. References

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

https://www.youtube.com/watch?v=YCzL96nL7j0

https://medium.com/@jyotikrisharma10/deberta-simplified-a-beginners-guide-with-real-example-ef890e6720da

BSDA2001P: Introduction to DL and GenAI Project

Hugging Face models(MiniLM & DeBERTa): huggingface.co

Weights & Biases: wandb.ai